计算机原理与编程基础

期末考试练习题

1．编写一个程序，要求用户输入一个分数，然后将其约分为最简分数。程序运行示例如下：

```
Enter a fraction: 6/12
In lowest terms: 1/2
```

分析：

- 为了把分数约分为最简分数，可以先计算分子和分母的最大公约数，然后分子分母都除以最大公约数；
- 求**最大公约数的Euclid算法**：分别让变量m和n存储两个数的值。如果n为0，那么停止操作，m中的值为最大公约数；反之，计算m除以n的余数，把n保存到m中，并把余数保存到n中。重复上述操作，每次都先判定n是否为0。

2．有些三位数称为“水仙花”数，即每位数的立方和等于该数本身。例如：

$$153 = 1^3 + 5^3 + 3^3$$

编写程序输出所有的水仙花数。

3．从屏幕上输入三个双精度数a，b，c，然后编程求解一元二次方程 $ax^2 + bx + c = 0$的根，然后把方程的根打印到屏幕上。

4．将输入的正整数分解质因数，例如：输入的正整数为120，那么输出为：120=2*2*2*3*5。

5．判断输入的正整数是否为质数。

6．输出100以内的所有质数。

提示：上述有关质数的题目，只需要在循环中检查不大于sqrt(n)的除数即可。

7．编写一个使字符串逆转的程序，例如使

"abcdef" -> "fedcba"

8．编写在磁盘上创建多级目录的函数

```
void CreateDirs(const char *strPath);
```

其中建单级文件夹的函数为 mkdir(char *str);目录分级的分隔符为'\\',最后一级可以有'\\',也可以没有 '\\'。

9．编写程序：从3个红球，5个白球，6个黑球中任意取出8个球，且其中必须有白球，输出所有可能的方案。

分析：设x,y,z分别是8个球中红、白和黑球的个数，题目转化为求出方程x+y+z=8在条件 $0 \le x \le 3,\ 1 \le y \le 5,\ 0 \le z \le 6$ 下的所有解。

10．由用户输入两个日期，日期的格式为：2017-01-01，然后计算出两天之间相隔的天数。

11．编写程序，用牛顿法计算正浮点数的平方根。算法如下：设x是用户输入的数，牛顿法需要先给出x平方根的猜测值y(例如y=1)，后续的猜测值通过计算y和x/y的平均值得到，迭代直至达到预定的精度10^{-6}即终止。

x	y	x/y	y和x/y的平均值
3	1	3	2
3	2	1.5	1.75
3	1.75	1.71429	1.73214
3	1.73214	1.73196	1.73205
3	1.73205	1.73205	1.73205

12．已知最古老的加密技术是凯撒加密。该方法把一条消息中的每个字母用字母表中固定距离之后的那个字母来替代(如果越过了Z，会绕回到字母表的起始位置。例如每个字母都用字母表中两个位置之后的字母代替，那么Y就被替代为A，Z就被替代为B)。编写程序用凯撒方法加密。用户输入待加密的消息和移位计数。

```
Enter message to be encrypted: Go ahead, make my day.
Enter shift amount [1-25]: 3
Encrypted message: Jr dkhdg, pdnh pb gdb.
```

提示：注意大小写字符！

13． 用变步长梯形求积法计算定积分 $T=\int_a^b f(x)dx$

- 算法：
- 首先用梯形公式计算：$T_1=\left[f(a)+f(b)\right]\cdot h/2$ ，其中h = b - a
- 然后用下面的公式递推

$$T_{2n}=T_n/2+\frac{h}{2}\sum_{i=0}^{n-1}f(x_i+0.5h)$$

$$2n\Rightarrow n,\quad 0.5h\Rightarrow h$$

- 直至 $|T_{2n}-T_n|<\varepsilon$ 为止。

- 设 a = 0, b = 1。函数为 $f(x)=e^{-x^2}$

14．下面的函数声明中，a和b是两个长度相同（都是n）的double型数组，这个函数的功能是计算数组a和b之间的相关系数，公式如下所示。请根据下面的函数声明方式完成该函数。

```
double CalcCorrelationCoef(double a[], double b[], int n);
```

相关系数的定义：

$$Cor(a,b)=\frac{\sum_{i=1}^{n}(a_i-\bar{a})\cdot(b_i-\bar{b})}{\sqrt{\sum_{i=1}^{n}(a_i-\bar{a})^2\cdot\sum_{i=1}^{n}(b_i-\bar{b})^2}} \quad ; \quad \bar{a}=\frac{\sum_{i=1}^{n}a_i}{n} \quad ; \quad \bar{b}=\frac{\sum_{i=1}^{n}b_i}{n}$$

提示：

$$\frac{\sum_{i=1}^{n}(a_i-\bar{a})\cdot(b_i-\bar{b})}{n}=\frac{\sum_{i=1}^{n}(a_ib_i-a_i\bar{b}-\bar{a}b_i+\bar{a}\bar{b})}{n}$$

$$=\frac{\sum_{i=1}^{n}a_ib_i}{n}-\frac{\sum_{i=1}^{n}a_i\bar{b}}{n}-\frac{\sum_{i=1}^{n}\bar{a}b_i}{n}+\frac{\sum_{i=1}^{n}\bar{a}\bar{b}}{n}=\overline{ab}-\bar{a}\cdot\bar{b}$$

$$\sqrt{\frac{\sum_{i=1}^{n}(a_i-\bar{a})^2}{n}}=\sqrt{\frac{\sum_{i=1}^{n}(a_i^2-2\bar{a}\cdot a_i+\bar{a}^2)}{n}}=\sqrt{\overline{a^2}-(\bar{a})^2}$$

15．下述函数需要从输入的字符串 const char* strPathName 中提取出文件的标题，然后存入输出参数 char* strTitle中。例如：输入的字符串为"d:\\Test\\sub01\\main.cpp"，那么函数返回的strTitle中存放的字符串为"main"。编程实现函数**GetFileTitle**。

```
void GetFileTitle(const char* strPathName, char* strTitle);
```

16．在指定的文本文件中搜索特定关键词出现的次数。其中文本文件为程序的输入参数，程序运行时需要输入关键词，然后程序统计出整个文件中关键词出现的次数。

程序运行示例如下：

```
F:\>F:\MyProgram\Prob16\Debug\Prob16.exe F:\MyProgram\data\Prob13.cpp
Search in file F:\MyProgram\data\Prob13.cpp
Enter keyword: double
Total 17 finds!
```

17．在指定的文本文件中搜索特定关键词，并按要求将其替换为另外一个关键词。程序运行时先输入要搜索的词，然后输入要替换的词，程序则遍历文件并完成词语的替换，替换后的内容另存入另一个新的文件，最后输出替换的次数。main函数如下所示：

```
void main() {
	char strSearch[] = "double";
	char strReplace[] = "float";
	char strSrcFile[] = "f:\\photo.cpp";
	char strTarFile[] = "f:\\photo-new.cpp";

	int nFind = ReplaceInFile(strSrcFile, strTarFile, strSearch, strReplace);
	printf("Total %d find and replace!\n", nFind);
}
```

18．下面是一个控制点文件的样本，第一行代表控制点数，后面每一行为一个控制点，其数据信息从左到右依次为：ID(控制点编号)，X，Y，Z坐标。要求编写一个函数读取控制点数据。

```
16
      7030001    6572229.763153    5569169.469701        294.347274
      7030002    6572465.847228    5569250.827822        275.506779
      7030003    6572459.367848    5569326.682539        286.819275
      7030004    6572337.376203    5569249.211691        300.499446
      7030005    6572350.106600    5569406.233226        277.019743
      7030006    6572234.730845    5569330.465243        276.536816
      7030007    6572231.521590    5569365.161337        291.512819
      7030008    6572118.742535    5569367.754632        289.931362
      7030009    6572469.204318    5569574.798347        269.948729
      7030010    6572346.635533    5569529.431538        287.589948
      7030011    6572355.274778    5569574.743085        269.152061
      7030012    6572349.954577    5569611.307013        269.687893
      7030013    6572237.015074    5569576.552589        269.932045
      7030014    6572239.539649    5569611.304469        269.744538
      7030015    6572117.740930    5569489.452039        268.521315
      7030016    6572122.987715    5569533.020047        268.038107
```

要求的结构和函数的声明如下：

```
typedef struct tagGCP{
        int ID;
        double x, y, z;
}GCP;
```

```
bool ReadGrdCtrlPtFile(const char* strFile, GCP** pGCP, int* nGCP);
```

或者

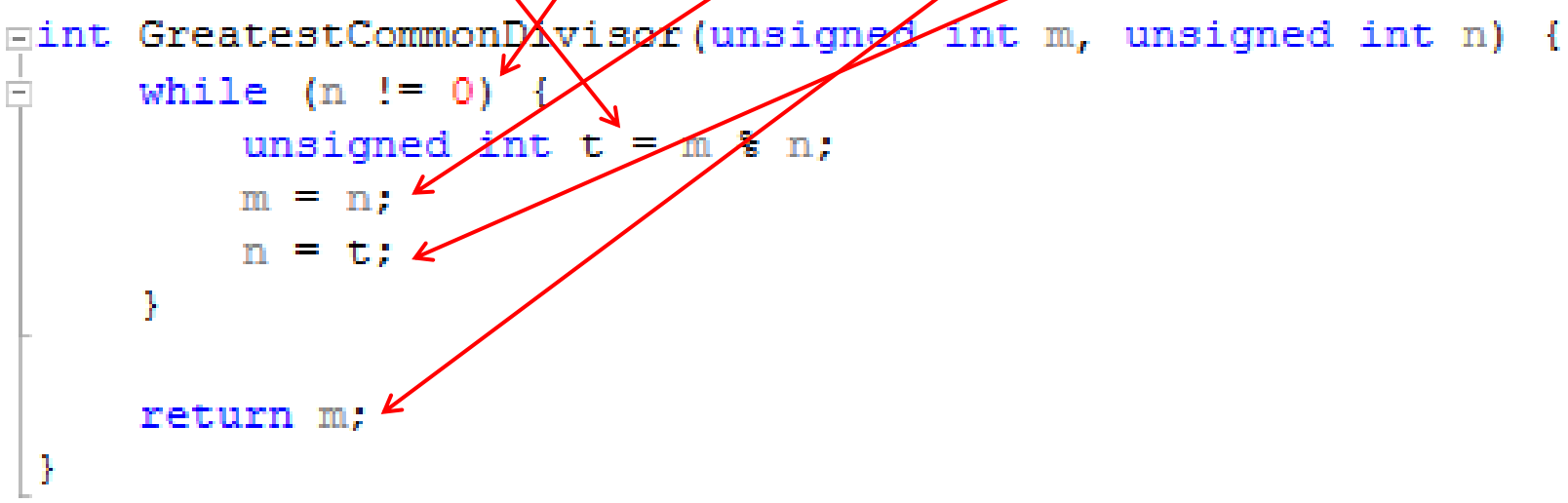
```
GCP* ReadGrdCtrlPtFile(const char* strFile, int* nGCP);
```

从文本文件中读取的内容另存到一个二进制文件。函数的声明方式如下：

```
bool WriteGCPAsBinFile(const char* strFile, GCP* pGCP, int nGCP);
```

习题解析：

1． 关键在于**最大公约数的Euclid算法：**分别让变量m和n存储两个数的值。如果n为0，那么停止操作，m中的值为最大公约数；反之，计算m除以n的余数，把n保存到m中，并把余数保存到n中。重复上述操作，每次都先判定n是否为0。



```
int GreatestCommonDivisor(unsigned int m, unsigned int n) {
    while (n != 0) {
        unsigned int t = m % n;
        m = n;
        n = t;
    }

    return m;
}
```

习题解析

2 ~ 7题略

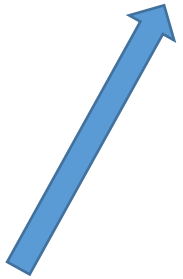
第8题：

```
void CreateDirectoryEx(const char* strPath) {
    char* p = strchr((char *)strPath, '\\');
    while (p) {
        char strTemp[256] = { 0 };
        memcpy(strTemp, strPath, (p - strPath) * sizeof(char));
        if (_access(strTemp, 0) != 0) _mkdir(strTemp);
        p = strchr(p + 1, '\\');
    }
    if (_access(strPath, 0) != 0) _mkdir(strPath);
}
```

- 字符串strTemp初始化为0的原因：0其实就是字符串的结束符'\0'！
- 所以，在memcpy这一句中拷贝完所需字符以后，无需再在末尾追加字符串结束符了。

习题解析

```
struct tagMyDate{
    unsigned int year, month, date;
};
typedef tagMyDate MyDate;
typedef tagMyDate* MyDatePtr;
```



第9题(略)

第10题：

```
typedef struct tagMyDate {
    unsigned int year, month, date;
}MyDate, *MyDatePtr;

const int DaysInMonth[] = {
    0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31,
    0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
};

bool IsLeapYear(unsigned int year) {
    return (((0 == year % 4) && (0 != year % 100)) || (0 == year % 400)) ? true : false;
}
```

采用查表方法获取每月的天数，避免复杂的逻辑判断

判断输入的年份是不是闰年

习题解析

- 比较两个日期的大小

```
int GetSign(int value) {
    if (value < 0) return -1;
    else if (value > 0) return 1;
    return 0;
}

int CompareDate(MyDatePtr t1, MyDatePtr t2) {
    int symbol = GetSign(t1->year - t2->year);
    if (symbol == 0) {
        symbol = GetSign(t1->month - t2->month);
        if (symbol == 0) {
            symbol = GetSign(t1->date - t2->date);
        }
    }
    return symbol;
}
```

习题解析

- 输入一个日期

```
MyDatePtr InputNewDate() {
    MyDatePtr t = new MyDate;
    printf("Please enter a date(xxxx/xx/xx): ");
    scanf("%d/%d/%d", &t->year, &t->month, &t->date);
    return t;
}
```

- 计算两个日期之间的天数

```
int GetDaysFromTheYear(MyDatePtr t) { ... }

int GetDifferentDays(MyDatePtr t1, MyDatePtr t2) { ... }
```

  - 先分别算出两个日期与当年1月1日之间的天数
  - 再算出两年1月1日之间相隔的天数
  - 最后再算出两个日期之间的天数

设计算法需要采用“分而治之”的策略！

习题解析

第11题：典型的迭代策略

```
double NewtonSqrt(double x) {
    double y = 1.0;
    double t = x;
    int nIter = 0;
    do {
        t = (y + x / y) / 2.0;
        if (fabs(t - y) < eps)
            break;

        y = t;
        nIter++;
    } while (nIter <= 20);

    return t;
}
```

- 两个变量存储两次迭代的计算值
- 设置迭代次数变量
- 设置最大迭代次数，避免无限循环
- 计算本次迭代的计算值
- 比较前后两次计算值的差，判定循环是否终止
- 更新上一次的迭代计算值
- 累加迭代次数

习题解析

第12题

查表！

```
char strCapital[]   = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
char strLowercase[] = "abcdefghijklmnopqrstuvwxyz";

bool CaesarEncryption(char* strInfo, int nShift) {
    if (nShift < 1 || nShift > 25)
        return false;

    char strCapTable[32] = { 0 }, strLowTable[32] = { 0 };
    memcpy(strCapTable, strCapital + nShift, (26 - nShift) * sizeof(char));
    memcpy(strLowTable, strLowercase + nShift, (26 - nShift) * sizeof(char));
    memcpy(strCapTable + 26 - nShift, strCapital, nShift * sizeof(char));
    memcpy(strLowTable + 26 - nShift, strLowercase, nShift * sizeof(char));

    char* p = strInfo;
    while (*p != '\0'){
        if (*p >= 'A' && *p <= 'Z') {
            int nCapId = (*p) - 'A';
            *p = strCapTable[nCapId];
        }
        else if (*p >= 'a' && *p <= 'z') {
            int nLowId = (*p) - 'a';
            *p = strLowTable[nLowId];
        }
        p++;
    }
}
```

习题解析

第13~15题（略）

第16题

```
int SearchKeywordInString(const char* strLine, const char* strWord) { ... }

int SearchKeywordInFile(const char* strFile, const char* strWord) { ... }
```

依然是分而治之：

- SearchKeywordInFile —— 在文件中查找，文件按行读！
- SearchKeywordInString —— 再每一行的字符串中查找！

第17题采用了相同的策略，不复赘述。

第18题主要考察如何将函数内部动态分配的内存传到函数外